THE OPEN-SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Student Name- Daman
Registration No- 24BAI10507
Slot- A24
Date of Submission- 31st March 2026

Chosen Software: Git

License: GNU General Public License v2 (GPL v2)

Introduction

This report presents a structured audit of Git — a distributed version control system created by Linus Torvalds in 2005. Git is one of the most widely used open-source tools in existence, underpinning virtually all modern software development. This audit examines Git's origin, its license and the freedoms it grants, its Linux footprint, the ecosystem it has created, and how it compares with proprietary alternatives.

Alongside this report, five shell scripts demonstrate practical Linux and shell scripting skills, each grounded in the open-source philosophy of transparency, automation, and sharing.

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

Before Git existed, the Linux kernel — one of the largest collaborative software projects in the world — was managed using a proprietary version control system called BitKeeper. BitKeeper was free to use for open-source projects, but it was not open source itself. The arrangement was always uncomfortable: a free-software community depending on a proprietary tool sat against the core values of the project.

In 2005, the relationship between the Linux kernel community and BitKeeper's owner, Larry McVoy, broke down. A developer in the community had attempted to reverse-engineer BitKeeper's protocol, which violated the terms of the free license. McVoy revoked the free access. The Linux kernel — and Torvalds himself — were suddenly without a version control system for one of the world's most important software projects.

The tools available at the time were inadequate. CVS and Subversion were centralised: all history lived on one server, and every significant operation required network access. They were also slow. Torvalds had very specific requirements: a system that was fast (able to handle the scale of the Linux kernel), truly distributed (every developer holds a complete copy of history), reliable (corruption should be cryptographically detectable), and completely free.

No existing tool met all four criteria. So Torvalds built one himself. He wrote the initial version of Git in approximately ten days in April 2005 and released it to the Linux kernel community almost immediately. The name 'Git' is British slang for a stupid or unpleasant person — Torvalds named it after himself, in a self-deprecating joke.

The design philosophy behind Git reflects the frustration that caused it to be built. Every repository contains the full history. Operations happen locally, without a server. Every object in the repository is identified by a SHA-1 hash, making corruption detectable and tampering evident. These were not arbitrary choices; they were direct responses to the failures of the tools that came before.

A2. The License — What It Actually Says

The Four Freedoms of Free Software

The Free Software Foundation defines four essential freedoms that a piece of software must grant to be considered truly free. These are:

- Freedom 0: The freedom to run the program for any purpose.
- Freedom 1: The freedom to study how the program works and to modify it.
- Freedom 2: The freedom to redistribute copies.
- Freedom 3: The freedom to distribute modified versions.

Git is licensed under the GNU General Public License version 2 (GPL v2). It grants all four freedoms. Anyone can download Git's source code, read it, modify it, and redistribute their changes. The only condition is that those changes must also be distributed under the GPL v2 — this is the 'copyleft' provision, which ensures that freedoms cannot be stripped away from downstream users.

Can a Company Take Git, Modify It, and Sell It Closed?

No. This is precisely what copyleft prevents. Under the GPL v2, if you distribute a modified version of Git, you must also make the source code of your modifications available under the same license. A company can build a commercial product that uses Git internally without releasing anything, but if they distribute a modified version of Git to customers, they must release the source.

This is different from permissive licenses like MIT or Apache 2.0, where you can take the code, modify it, and distribute a closed-source product with no obligation to share your changes.

GPL vs MIT — The Core Difference

The MIT license is a permissive license. It grants almost all freedoms with almost no strings attached. You can take MIT-licensed code, incorporate it into a proprietary product, and never share your changes. The only requirement is attribution — you must keep the copyright notice. This makes MIT very friendly to commercial use.

The GPL is a copyleft license. It grants the same freedoms but requires that any distributed derivative also be GPL-licensed. This is sometimes called 'viral' because the license requirement spreads to derivative works. For a public library or a tool that will be embedded in many different contexts, MIT might be the better choice. For a tool you want to ensure remains forever open — where you want to prevent corporations from forking it and closing the fork — GPL is more appropriate.

For Git specifically, Torvalds chose GPL v2 deliberately. Git is infrastructure. He wanted to ensure it would never be taken private. Given the history — watching proprietary version control fail the Linux community — the choice makes complete sense.

Free as in Free Beer vs Free as in Freedom

The phrase 'free as in free beer' refers to price — the software costs nothing to acquire. 'Free as in freedom' refers to liberty — the software grants you the right to study, modify, and share it. The distinction matters enormously in practice.

Git is free in both senses. You pay nothing for it. But more importantly, you are free to examine exactly how it stores your repository, free to fix a bug that affects your workflow, and free to contribute that fix back. When BitKeeper was free to use, it was only free as in free beer. The Linux community had no right to modify or redistribute it. That dependency was the vulnerability that created the crisis in 2005. Git was built specifically to never repeat that mistake.

A3. The Ethics of Open Source

Open source presents genuine ethical questions that do not have easy answers. Three in particular are worth examining seriously.

Should All Software Be Open Source?

The argument for universal open source is compelling: software is increasingly infrastructure. The tools that run hospitals, financial systems, and democratic processes should be auditable by anyone, not held as trade secrets by corporations. When software is opaque, vulnerabilities can hide indefinitely. When it is open, the community acts as a distributed security review.

The argument against is equally serious. Open-source development requires sustained effort. Without revenue models, many critical projects are maintained by volunteers who burn out. The OpenSSL vulnerability Heartbleed in 2014 exposed code that had been 'open' for years but barely reviewed, because no one was paid to review it. Proprietary software,

with commercial incentives, sometimes produces more secure and reliable results than underfunded open-source alternatives. The answer is probably not that all software should be open, but that more infrastructure-critical software should be, with sustainable funding models attached.

Is It Ethical to Profit from Community Labor?

Companies like Google, Microsoft, and Meta run enormous portions of their infrastructure on the Linux kernel, Git, and thousands of other open-source libraries. They profit massively from code they did not write and did not pay for. Is that ethical?

There is a principled case that it is — because the GPL and similar licenses explicitly permit commercial use. The developers who wrote that code chose a license that allows it. Companies like Red Hat built a business model around supporting and contributing back to open source, which funds the development of the tools they depend on. The question becomes more uncomfortable when a company takes without giving back: using the community's work to build proprietary advantage while contributing nothing to the commons. This is technically permitted but represents a genuine ethical tension.

Standing on the Shoulders of Giants

The phrase, attributed to Isaac Newton, describes how each generation of knowledge builds on the last. In software, this is almost entirely mediated by open source. Git itself depends on zlib (data compression), OpenSSL (cryptography), and libc (the C standard library). Without those libraries — themselves open source — Git could not exist in its current form.

Open source enables this accumulation of knowledge in a way that proprietary software cannot. When a company closes its code, that knowledge dies with the product or becomes inaccessible. When code is open, every generation can stand on it. The argument that this undermines original innovation misunderstands how innovation actually works — almost no significant software is built from nothing. It is always built from prior art. Open source simply makes that prior art accessible and legally shareable.

Part B — Linux Footprint

Installation Method

Git is available through virtually every Linux package manager. On Debian and Ubuntu-based systems, installation is handled with:

```
sudo apt update
sudo apt install git
```

On Red Hat, Fedora, and CentOS systems:

```
sudo dnf install git
```

For users who need a very recent version, Git can be compiled from source by downloading the tarball from git-scm.com and running the standard configure/make/make install sequence. Most users will never need this, as distribution packages are kept reasonably current.

Key Directories

Path	Purpose
/usr/bin/git	The main Git binary — what executes when you type 'git'
/usr/lib/git-core/	Git helper executables (git-commit, git-merge, etc.)
/usr/share/git-core/	Architecture-independent data files and templates
/etc/gitconfig	System-wide Git configuration (applies to all users)
~/.gitconfig	Per-user configuration (name, email, aliases, preferences)
.git/ (per repo)	Repository data: objects, refs, config, HEAD, index

User and Group

Git runs as whichever user invokes it — there is no dedicated 'git' system user for local operations. This is by design: Git is a user-space tool, not a service. The files inside a .git/ directory are owned by the user who initialised the repository.

On servers running GitLab or Gitea (web frontends for Git), a dedicated 'git' system user is created. This user owns the repository files and the application process runs under that account. Limiting the service to a low-privilege user means that even if the application is compromised, the attacker gains access only to the git user's files and permissions, not root.

This principle — running services as minimally-privileged dedicated users — is a core Linux security practice that Git's server deployments follow correctly.

Service Management

Git itself is not a persistent service. However, the SSH daemon (sshd) and any Git web frontend (such as GitLab) are managed as systemd services. Common commands:

```
# Check status of the SSH service (used by Git over SSH)
systemctl status sshd

# For GitLab (if installed)
sudo gitlab-ctl start
sudo gitlab-ctl stop
sudo gitlab-ctl restart
```

Update Model

Security updates to Git arrive through the standard distribution update mechanism. When the Git security team identifies a vulnerability, they coordinate a release, and within days the patched version appears in distribution repositories. On a Debian/Ubuntu system:

```
sudo apt update && sudo apt upgrade git
```

This is the open-source security update model in action: the source code is publicly visible, vulnerabilities can be reported to the security team, the fix is developed in the open, and the patch is distributed through a trusted channel. The transparency of the process is itself a security property — anyone can verify that the fix actually addresses the reported vulnerability.

Part C — The FOSS Ecosystem

Dependencies

Git is written primarily in C and depends on several foundational open-source libraries:

- zlib: Used for compressing objects stored in the repository. Compression reduces disk usage and transfer time.
- OpenSSL / libssl: Used for SHA-1 hashing (Git's object identification scheme) and HTTPS transport.
- libcurl: Handles HTTP and HTTPS communication for remote operations like git push and git pull.
- Perl: Several Git helper scripts are written in Perl, particularly for log formatting and certain hooks.
- GNU C Library (glibc): The foundational C runtime on Linux systems.

What Git Has Enabled and Inspired

Git's impact on the software development ecosystem is almost impossible to overstate. It has directly enabled:

- GitHub (2008): A platform for hosting Git repositories with collaboration features. As of 2024, it hosts over 330 million repositories and is the central hub of open-source development globally.
- GitLab: An open-source alternative to GitHub that can be self-hosted. Used extensively by companies and governments that need on-premise control.
- Gitea and Forgejo: Lightweight, self-hosted Git services for smaller teams and communities.
- CI/CD pipelines: GitHub Actions, GitLab CI, and Jenkins all build on Git's event model — every push, merge request, and tag can trigger automated workflows.
- Git LFS (Large File Storage): An extension for managing large binary files in repositories.

Community and Governance

The Git project is governed by Junio C Hamano, who has been the maintainer since Torvalds handed it off in 2005. Development happens on the git@vger.kernel.org mailing list — one of the oldest and most active developer mailing lists in open source. Patches are submitted, reviewed, and merged entirely through email, which reflects the project's Unix roots.

There is no corporate owner of Git. While Microsoft owns GitHub (the platform), Git the tool is an independent project with a governance model built on meritocracy and consensus. The Git community is notably conservative about breaking changes, prioritising backward compatibility above almost everything else.

Git and the LAMP Stack

Git is not part of the traditional LAMP stack (Linux, Apache, MySQL, PHP), but it is foundational to how LAMP applications are developed and deployed. In modern DevOps practice, Git serves as the source of truth: every change to a web application, configuration file, or database schema begins with a git commit. Deployment pipelines pull from Git repositories. Infrastructure-as-code tools like Ansible and Terraform use Git for version

control. Git is, in a sense, the tool that sits above and behind every layer of the modern web stack.

Part D — Open Source vs Proprietary

Comparison: Git vs Perforce Helix Core

The closest enterprise proprietary alternative to Git is Perforce Helix Core, used extensively in game development, finance, and large binary asset pipelines.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free. No licensing fees. Self-host at zero cost or use GitHub/GitLab.	Commercial licensing. Costs scale with users and storage. Can be significant for large teams.
Security (auditability)	Full source code is public. Any security researcher can audit it. CVEs are publicly tracked.	Closed source. Security depends entirely on Perforce's internal team. No independent audit possible.
Support and reliability	Community support via mailing lists and Stack Overflow. Commercial support via GitHub Enterprise or GitLab.	Professional support included in licensing. SLAs available. Better for enterprises with compliance requirements.
Freedom to modify	Complete freedom. Fork it, patch it, build on it — under GPL v2.	None. You use what Perforce ships.
Community vs corporate	Community-governed. No single company controls the tool's direction.	Perforce Inc. controls all decisions. Product direction follows commercial incentives.
Large binary files	Weak by default; requires Git LFS extension with additional setup.	Designed for it. Game studios with hundreds of GB of assets prefer Perforce for this reason.

Verdict

For the overwhelming majority of software development projects — web applications, open-source libraries, research code, student projects, startup codebases — Git is unambiguously the right choice. It is free, universally understood, supported by an enormous ecosystem of tools, and has been hardened by use in the largest software projects in history.

Perforce has genuine advantages in very specific niches: large binary asset pipelines (game development with multi-gigabyte asset files), and enterprise environments with complex compliance requirements where vendor support contracts are mandatory. In those contexts, the cost and closed-source nature are acceptable trade-offs for the technical capabilities.

Would I contribute to Git? Absolutely. The Git mailing list is one of the most technically rigorous contributor communities in open source. Contributing to Git is a meaningful way to improve infrastructure that millions of developers depend on every day. The barrier to entry is high — patches are reviewed in detail and standards are demanding — but that rigour is exactly why Git has remained reliable for nearly twenty years.

Shell Scripts

Script 1 — System Identity Report

This script acts as a welcome screen that introduces the Linux system and the open-source audit context. It collects key system details such as the Linux distribution name, kernel version, logged-in username, home directory, system uptime, and current date and time, then displays them in a neatly formatted output. It also prints the GPL v2 license information for the OS and a note about Git being the chosen audit software. The shell scripting concepts used include **variables** to store student name and software choice, **command substitution** using $() to capture live output from commands like uname -r, whoami, uptime -p, and date, **pipes and cut** to extract the distro name from /etc/os-release, and **echo** with formatted strings to produce readable, aligned output.

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Daman | Reg No: 24BAI10507
# Course: Open Source Software
# ─── Variables ──────────────────────────────────────────────
STUDENT_NAME="Daman"
SOFTWARE_CHOICE="Git"
# Gather system information using command substitution
KERNEL=$(uname -r)                    # Linux kernel version
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')  # Distro name
USER_NAME=$(whoami)                   # Currently logged-in user
HOME_DIR=$HOME                        # Home directory of current user
UPTIME=$(uptime -p)                   # Human-readable uptime
CURRENT_DATE=$(date '+%d %B %Y, %I:%M %p')   # Formatted date and time

# ─── Display Header ─────────────────────────────────────────
echo "==================================================="
echo "        Open Source Audit — $STUDENT_NAME"
echo "        Chosen Software  : $SOFTWARE_CHOICE"
echo "==================================================="
echo ""

# ─── System Information ─────────────────────────────────────
echo "  Linux Distribution : $DISTRO"
echo "  Kernel Version     : $KERNEL"
echo "  Logged-in User     : $USER_NAME"
echo "  Home Directory     : $HOME_DIR"
echo "  System Uptime      : $UPTIME"
echo "  Current Date/Time  : $CURRENT_DATE"
echo ""
# ─── License Information ────────────────────────────────────
# The Linux kernel is licensed under GPL v2 (GNU General Public License version 2)
# This means the OS itself is free and open-source software
echo "  OS License         : GNU General Public License v2 (GPL v2)"
```

```
echo "  This means you are free to run, study, modify, and"
echo "  redistribute this operating system and its source code."
echo ""
# ── Git-specific note ──────────────────────────────────────
# Git is also licensed under GPL v2 — same as the Linux kernel
echo "  Audit Target       : $SOFTWARE_CHOICE (Licensed under GPL v2)"
echo "  Git was created by Linus Torvalds in 2005 when the"
echo "  proprietary BitKeeper VCS revoked free access to the"
echo "  Linux kernel development community."
echo ""
echo "==================================================="
echo "  'Free as in freedom, not just free as in beer.'"
echo "           — Richard Stallman, GNU Project"
echo "===================================================="
```

Output:

```
(base) damanmaccy@Damans-MacBook-Air oss-audit % chmod +x *.sh
[./script1_system_identity.sh
cat: /etc/os-release: No such file or directory
uptime: illegal option -- p
usage: uptime
====================================================
        Open Source Audit — Daman
        Chosen Software   : Git
====================================================

  Linux Distribution :
  Kernel Version     : 25.2.0
  Logged-in User     : damanmaccy
  Home Directory     : /Users/damanmaccy
  System Uptime      :
  Current Date/Time  : 27 March 2026, 02:55 PM

  OS License         : GNU General Public License v2 (GPL v2)
(base) damanmaccy@Damans-MacBook-Air oss-audit %
```

Script 2 — FOSS Package Inspector

This script checks whether Git is installed on the system and displays its package details including version, license, and summary. It works across both RPM-based systems (Fedora, RHEL) and DEB-based systems (Ubuntu, Debian) by detecting which package manager is available. After displaying package info, it prints a one-line philosophy note about the selected package. The shell scripting concepts used include **if-then-else** for checking whether the package is installed and which package manager is present, **command -v** to detect available tools, **pipes with grep** using -E to filter specific fields from package output, and a **case statement** that matches the package name to a philosophy message, with a default * case as fallback for unrecognised packages.

Code:

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: DAMAN | Reg No: 24BAI10507
# Course: Open Source Software
# ─── Variable ──────────────────────────────────────────────────
PACKAGE="git"   # The open-source package we are auditing
# ─── Check if package is installed ──────────────────────────
# Try rpm first (for Fedora/RHEL/CentOS), then dpkg (for Debian/Ubuntu)
echo "==================================================="
echo "        FOSS Package Inspector — $PACKAGE"
echo "==================================================="
echo ""
if command -v rpm &>/dev/null; then
    # RPM-based system (Fedora, RHEL, CentOS)
    if rpm -q $PACKAGE &>/dev/null; then
        echo "  ✔ '$PACKAGE' is installed on this system (RPM)."
        echo ""
        echo "  Package Details:"
        echo "  ----------------"
        # Use grep with pipe to extract only the important fields
        rpm -qi $PACKAGE | grep -E 'Name|Version|License|Summary|URL'
    else
        echo "  ✘ '$PACKAGE' is NOT installed."
        echo "  To install, run: sudo dnf install git"
    fi
elif command -v dpkg &>/dev/null; then
    # DEB-based system (Ubuntu, Debian)
   if dpkg -s $PACKAGE &>/dev/null 2>&1; then
        echo "  ✔ '$PACKAGE' is installed on this system (DEB)."
        echo ""
        echo "  Package Details:"
        echo "  ----------------"
        # dpkg -s gives full package status and info
        dpkg -s $PACKAGE | grep -E 'Package|Version|License|Description'
    else
        echo "  ✘ '$PACKAGE' is NOT installed."
        echo "  To install, run: sudo apt install git"
    fi

else
    # Fallback: just check if the git command is available
    if command -v git &>/dev/null; then
        echo "  ✔ '$PACKAGE' is available on this system."
        echo "  Version: $(git --version)"
    else
        echo "  ✘ '$PACKAGE' is NOT installed or not in PATH."
```

```
    fi
fi

echo ""
# ─── Philosophy note using case statement ────────────────────
# Each package name maps to a one-line philosophy note about its OSS values
echo "  OSS Philosophy Note:"
echo "  --------------------"
case $PACKAGE in
    git)
        echo "  Git: Born from necessity — Linus built it in 2 weeks when"
        echo "  proprietary tools failed the Linux community. GPL v2 ensures"
        echo "  it stays free for every developer, forever."
        ;;
    httpd|apache2)
        echo "  Apache: The web server that built the open internet."
        echo "  Apache 2.0 license lets companies use it freely while"
        echo "  still protecting contributors' work."
        ;;
    mysql)
        echo "  MySQL: Open source at the heart of millions of apps."
        echo "  Its dual GPL/commercial license sparked important debates"
        echo "  about what 'open source' truly means for businesses."
        ;;
    vlc)
        echo "  VLC: Built by students in Paris who just wanted to watch"
        echo "  video on their campus network. LGPL ensures it stays open"
        echo "  and plays anything, on any platform."
        ;;
    firefox)
        echo "  Firefox: A nonprofit's answer to browser monopoly."
        echo "  MPL 2.0 lets anyone contribute while keeping the core"
        echo "  code open — a browser truly built for users, not profit."
        ;;
    python3|python)
        echo "  Python: A language shaped entirely by its community."
        echo "  The PSF license is one of the most permissive in OSS —"
        echo "  use it anywhere, build anything, share what you learn."
        ;;
    *)
        # Default case if the package name doesn't match any known entry
        echo "  '$PACKAGE' is an open-source tool. Check its LICENSE file"
         echo "  to understand what freedoms it grants you."
         ;;
esac
```

```
echo ""
echo "==================================================="
```

Output:

```
(base) damanmaccy@Damans-MacBook-Air oss-audit % ./script2_package_inspector.sh
===================================================
        FOSS Package Inspector — git
===================================================

  ✔ 'git' is available on this system.
  Version: git version 2.39.5 (Apple Git-154)

  OSS Philosophy Note:
  --------------------
  Git: Born from necessity — Linus built it in 2 weeks when
  proprietary tools failed the Linux community. GPL v2 ensures
  it stays free for every developer, forever.

===================================================
(base) damanmaccy@Damans-MacBook-Air oss-audit %
```

Script 3 — Disk and Permission Auditor

This script loops through a predefined list of important Linux system directories and reports the permissions, owner, group, and disk usage of each one. It also performs a dedicated check on Git's configuration file locations — the system-wide /etc/gitconfig and the user-level ~/.gitconfig — explaining why their permissions matter for security. The shell scripting concepts used include a **for loop** iterating over an array of directory paths, an **if-else** block using -d to check if a directory exists before inspecting it, **ls -ld with awk** to extract permission and ownership fields, **du -sh with cut** to get human-readable directory sizes, and separate **file existence checks** using -f and -d flags for the Git config files.

Code:

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: DAMAN| Reg No: 24BAI10507
# Course: Open Source Software
# ─── List of important system directories to audit ──────────
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
# ─── Header ──────────────────────────────────────────────
echo "==================================================="
echo "         Disk and Permission Auditor"
echo "==================================================="
echo ""
echo "  Directory Audit Report"
echo "  -----------------------"
# ─── For loop through each directory ─────────────────────
for DIR in "${DIRS[@]}"; do
    # Check if the directory actually exists before inspecting it
    if [ -d "$DIR" ]; then
        # Extract permissions, owner, and group using awk on ls -ld output
```

```
        # ls -ld gives a single line for the directory itself (not its contents)
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')    # e.g. drwxr-xr-x
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')    # e.g. root
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')    # e.g. root

        # du -sh gives human-readable size; 2>/dev/null suppresses permission errors
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        echo "  Directory : $DIR"
        echo "  Size      : $SIZE"
        echo "  Perms     : $PERMS  |  Owner: $OWNER  |  Group: $GROUP"
        echo "  ------------------------------------------------"
    else
        # Directory doesn't exist on this system — print a note
        echo "  [SKIP] $DIR does not exist on this system."
        echo "  ------------------------------------------------"
    fi
done

echo ""
# ─── Git-specific config directory check ────────────────────
# Git stores its system-wide config at /etc/gitconfig
# and user-level config at ~/.gitconfig
echo "  Git Configuration Files"
echo "  -----------------------"

GIT_SYSTEM_CONFIG="/etc/gitconfig"
GIT_USER_CONFIG="$HOME/.gitconfig"
GIT_GLOBAL_DIR="$HOME/.config/git"

# Check system-wide gitconfig
if [ -f "$GIT_SYSTEM_CONFIG" ]; then
    PERMS=$(ls -l "$GIT_SYSTEM_CONFIG" | awk '{print $1, $3, $4}')
    echo "  ✔ System config  : $GIT_SYSTEM_CONFIG"
    echo "    Permissions    : $PERMS"
else
    echo "  ✘ System config  : $GIT_SYSTEM_CONFIG not found."
fi

echo ""

# Check user-level gitconfig
if [ -f "$GIT_USER_CONFIG" ]; then
    PERMS=$(ls -l "$GIT_USER_CONFIG" | awk '{print $1, $3, $4}')
    echo "  ✔ User config    : $GIT_USER_CONFIG"
    echo "    Permissions    : $PERMS"
```

```
else
    echo "  ✘ User config    : $GIT_USER_CONFIG not found."
    echo "    (Run 'git config --global user.name \"Name\"' to create it.)"
fi
echo ""
# Check ~/.config/git directory (newer Git versions use this)
if [ -d "$GIT_GLOBAL_DIR" ]; then
    PERMS=$(ls -ld "$GIT_GLOBAL_DIR" | awk '{print $1, $3, $4}')
    echo "  ✔ Global git dir : $GIT_GLOBAL_DIR"
    echo "    Permissions    : $PERMS"
else
    echo "  ✘ Global git dir : $GIT_GLOBAL_DIR not found."
fi
echo ""
echo "==================================================="
echo "  Why permissions matter:"
echo "  Config files contain user identity info used in"
echo "  every commit. Wrong permissions could let other"
echo "  users on the same system read or modify them."
echo "==================================================="
```

Output:

```
==================================================
(base) damanmaccy@Damans-MacBook-Air oss-audit % nano script3_disk_permission_auditor.sh
(base) damanmaccy@Damans-MacBook-Air oss-audit % chmod +x *.sh
(base) damanmaccy@Damans-MacBook-Air oss-audit % ./script3_disk_permission_auditor.sh
==================================================
         Disk and Permission Auditor
==================================================

  Directory Audit Report
  ----------------------
  Directory : /etc
  Size      :   0B
  Perms     : lrwxr-xr-x@  |  Owner: root  |  Group: wheel
  ----------------------------------------------
  Directory : /var/log
  Size      :  91M
  Perms     : drwxr-xr-x  |  Owner: root  |  Group: wheel
  ----------------------------------------------
  Directory : /home
  Size      :   0B
  Perms     : lrwxr-xr-x  |  Owner: root  |  Group: wheel
  ----------------------------------------------
  Directory : /usr/bin
  Size      :  80M
  Perms     : drwxr-xr-x  |  Owner: root  |  Group: wheel
  ----------------------------------------------
  Directory : /tmp
  Size      :   0B
  Perms     : lrwxr-xr-x@  |  Owner: root  |  Group: wheel
  ----------------------------------------------

  Git Configuration Files
  -----------------------
  ✘ System config  : /etc/gitconfig not found.

  ✔ User config    : /Users/damanmaccy/.gitconfig
    Permissions    : -rw-r--r-- damanmaccy staff

  ✘ Global git dir : /Users/damanmaccy/.config/git not found.

==================================================
  Why permissions matter:
  Config files contain user identity info used in
  every commit. Wrong permissions could let other
  users on the same system read or modify them.
==================================================
```

Script 4 — Log File Analyzer

This script reads a log file line by line, searches for a user-specified keyword, counts how many lines contain that keyword, and prints the last five matching lines as a summary. It accepts the log file path and keyword as command-line arguments, defaulting to "error" if no

keyword is provided. It also includes a retry mechanism that re-checks the file up to three times if it is found to be empty. The shell scripting concepts used include **command-line arguments** via $1 and $2 with a default value using ${2:-"error"}, a **while-read loop** with IFS= and -r to safely read the file line by line, an **if-then** inside the loop to check each line for the keyword using grep -iq, a **counter variable** incremented with $((COUNT + 1)), a **do-while style retry loop** simulated using while true with a break condition, and **grep piped into tail** to extract the last five matching lines.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: DAMAN | Reg No: 24BAI10507
# Course: Open Source Software
# ─── Arguments ──────────────────────────────────────────────
LOGFILE=$1                  # First argument: path to the log file
KEYWORD=${2:-"error"}       # Second argument: keyword to search (default: "error")
COUNT=0                     # Counter for matching lines
MAX_RETRIES=3               # Max number of retries if file is empty
ATTEMPT=1                   # Current retry attempt number

# ─── Validate that a file argument was provided ──────────────
if [ -z "$LOGFILE" ]; then
    echo "  Usage: $0 <logfile> [keyword]"
    echo "  Example: $0 /var/log/syslog error"
    exit 1
fi

# ─── Check that the file exists ──────────────────────────────
if [ ! -f "$LOGFILE" ]; then
    echo "  Error: File '$LOGFILE' not found. Please check the path."
    exit 1
fi

# ─── do-while style retry loop if file is empty ──────────────
# Bash doesn't have a native do-while, so we simulate it with
# a while loop that always runs at least once using a flag.
while true; do
    # Check if the file has any content (size > 0)
    if [ -s "$LOGFILE" ]; then
        # File has content — break out of retry loop and proceed
        break
    else
        echo "  Warning: '$LOGFILE' appears to be empty. (Attempt $ATTEMPT of $MAX_RETRIES)"

        # If we've hit the max retries, exit gracefully
        if [ "$ATTEMPT" -ge "$MAX_RETRIES" ]; then
            echo "  File is still empty after $MAX_RETRIES attempts. Exiting."
            exit 1
        fi
```

```
        # Increment attempt counter and wait 2 seconds before retrying
        ATTEMPT=$((ATTEMPT + 1))
        sleep 2
    fi
done

# ─── Header ──────────────────────────────────────────────────
echo "==================================================="
echo "           Log File Analyzer"
echo "==================================================="
echo "  Log File : $LOGFILE"
echo "  Keyword  : '$KEYWORD'"
echo ""

# ─── Read file line by line using while-read loop ────────────
# IFS= preserves leading/trailing whitespace in each line
# -r flag prevents backslash interpretation
while IFS= read -r LINE; do
    # Check if the current line contains the keyword (case-insensitive with -i)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment match counter
    fi
done < "$LOGFILE"   # Redirect file into the while loop (not a subshell)

# ─── Print summary ───────────────────────────────────────────
echo "  Result: Keyword '$KEYWORD' was found $COUNT time(s) in $LOGFILE"
echo ""

# ─── Print last 5 matching lines ─────────────────────────────
# Combine grep (to filter matches) and tail (to get last 5 of those matches)
echo "  Last 5 lines containing '$KEYWORD':"
echo "  -------------------------------------"

MATCHES=$(grep -i "$KEYWORD" "$LOGFILE")

if [ -z "$MATCHES" ]; then
    # No matches found — inform the user
    echo "  (No lines matched '$KEYWORD' in this file.)"
else
    # Print last 5 matching lines using tail
    echo "$MATCHES" | tail -5
fi

echo ""
echo "==================================================="
```

Output:

```
(base) damanmaccy@Damans-MacBook-Air oss-audit % nano script4_log_analyzer.sh
(base) damanmaccy@Damans-MacBook-Air oss-audit % chmod +x *.sh
(base) damanmaccy@Damans-MacBook-Air oss-audit % ./script4_log_analyzer.sh /var/log/system.log error
==================================================
           Log File Analyzer
==================================================
  Log File : /var/log/system.log
  Keyword  : 'error'

  Result: Keyword 'error' was found 0 time(s) in /var/log/system.log

  Last 5 lines containing 'error':
  ------------------------------------
  (No lines matched 'error' in this file.)

==================================================
```

Script 5 — Open Source Manifesto Generator

This script interactively asks the user three questions about their open-source values, then uses their answers to compose a personalised philosophy statement which is saved to a .txt file named after the current user. It also demonstrates the concept of aliases through a comment, showing how Git users typically create shell aliases to speed up their workflow. The shell scripting concepts used include **read with -p** to capture interactive user input, **input validation** using -z to check for empty responses, **string concatenation** by embedding variables like $TOOL, $FREEDOM, and $BUILD directly into echo strings, **file writing** using > to create the file and >> to append each line, the **date command** with a custom format string for the timestamp, and **cat** to read and display the saved file back to the terminal.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: DAMAN| Reg No: 24BAI10507
# Course: Open Source Software
# ─── Header ──────────────────────────────────────────────────
echo "==================================================="
echo "        Open Source Manifesto Generator"
echo "         Powered by Git & GNU Bash"
echo "==================================================="
echo ""
echo "  Answer three questions to generate your personal"
echo "  open-source philosophy statement."
echo ""

# ─── Interactive user input using read ──────────────────────
# -p flag displays the prompt on the same line as the input cursor
read -p "  1. Name one open-source tool you use every day: " TOOL
```

```
echo ""

read -p "  2. In one word, what does 'freedom' mean to you? " FREEDOM
echo ""

read -p "  3. Name one thing you would build and share freely: " BUILD
echo ""

# ─── Validate that the user actually entered something ──────
if [ -z "$TOOL" ] || [ -z "$FREEDOM" ] || [ -z "$BUILD" ]; then
    echo "  Error: All three answers are required. Please re-run the script."
    exit 1
fi

# ─── Date and output file setup ──────────────────────────────
DATE=$(date '+%d %B %Y')                      # Formatted date e.g. 26 March 2026
OUTPUT="manifesto_$(whoami).txt"              # Output filename includes the username

# ─── String concatenation to build the manifesto ─────────────
# Each echo >> appends a new line to the output file
# The > operator is used first to create/overwrite the file cleanly

# Write the manifesto title and metadata to the file
echo "==================================================" > "$OUTPUT"
echo "         MY OPEN SOURCE MANIFESTO"                  >> "$OUTPUT"
echo "  Generated on : $DATE"                             >> "$OUTPUT"
echo "  Author       : $(whoami)"                         >> "$OUTPUT"
echo "==================================================" >> "$OUTPUT"
echo ""                                                   >> "$OUTPUT"

# String concatenation happens naturally as variables are embedded in the echo strings
echo "  Every day, I rely on $TOOL — a tool that exists"         >> "$OUTPUT"
echo "  not because someone wanted to profit from it, but"       >> "$OUTPUT"
echo "  because a community believed that $FREEDOM should"       >> "$OUTPUT"
echo "  not be locked behind a paywall or a proprietary wall."   >> "$OUTPUT"
echo ""                                                           >> "$OUTPUT"
echo "  Open source taught me that the most powerful things"     >> "$OUTPUT"
echo "  are built in the open. That is why I commit to one day"  >> "$OUTPUT"
echo "  sharing $BUILD freely — so that someone else can build"  >> "$OUTPUT"
echo "  on top of it, improve it, and pass it forward."          >> "$OUTPUT"
echo ""                                                           >> "$OUTPUT"
echo "  Like Git, which Linus Torvalds built and gave to the"    >> "$OUTPUT"
echo "  world under the GPL, I believe that knowledge grows"     >> "$OUTPUT"
echo "  stronger when it is shared, not hoarded."                >> "$OUTPUT"
echo ""                                                           >> "$OUTPUT"
echo "  Freedom in software means $FREEDOM — and that is"        >> "$OUTPUT"
```

```
echo "  worth protecting, contributing to, and passing on."      >> "$OUTPUT"
echo ""                                                           >> "$OUTPUT"
echo "==================================================" >> "$OUTPUT"
echo "  'Given enough eyeballs, all bugs are shallow.'"  >> "$OUTPUT"
echo "           — Eric S. Raymond, The Cathedral"       >> "$OUTPUT"
echo "             and the Bazaar"                       >> "$OUTPUT"
echo "==================================================" >> "$OUTPUT"

# ─── Confirm save and display the manifesto ─────────────────
echo "  ✔ Manifesto saved to: $OUTPUT"
echo ""
echo "  Here is your manifesto:"
echo ""

# cat reads and prints the file content to the terminal
cat "$OUTPUT"
```

Output:

```
(base) damanmaccy@Damans-MacBook-Air oss-audit % nano script5_manifesto_generator.sh
(base) damanmaccy@Damans-MacBook-Air oss-audit % chmod +x *.sh
(base) damanmaccy@Damans-MacBook-Air oss-audit % ./script5_manifesto_generator.sh
==================================================
       Open Source Manifesto Generator
         Powered by Git & GNU Bash
==================================================

  Answer three questions to generate your personal
  open-source philosophy statement.

  1. Name one open-source tool you use every day: git

  2. In one word, what does 'freedom' mean to you? travelling and enjoyment

  3. Name one thing you would build and share freely: a version control tool

  ✔ Manifesto saved to: manifesto_damanmaccy.txt

  Here is your manifesto:

==================================================
         MY OPEN SOURCE MANIFESTO
  Generated on : 27 March 2026
  Author       : damanmaccy
==================================================

  Every day, I rely on git — a tool that exists
  not because someone wanted to profit from it, but
  because a community believed that travelling and enjoyment should
  not be locked behind a paywall or a proprietary wall.

  Open source taught me that the most powerful things
  are built in the open. That is why I commit to one day
  sharing a version control tool freely — so that someone else can build
  on top of it, improve it, and pass it forward.

  Like Git, which Linus Torvalds built and gave to the
  world under the GPL, I believe that knowledge grows
  stronger when it is shared, not hoarded.

  Freedom in software means travelling and enjoyment — and that is
  worth protecting, contributing to, and passing on.

==================================================
  'Given enough eyeballs, all bugs are shallow.'
           — Eric S. Raymond, The Cathedral
             and the Bazaar
==================================================
```

Conclusion

Git was not built to win a market. It was built in response to a specific failure — the collapse of trust in a proprietary tool that a community had been forced to depend on. That origin story is the clearest possible illustration of why open-source licensing matters: dependency on proprietary software is a vulnerability, not just philosophically but practically.

Studying Git's license reveals that 'free' means far more than free of charge. The GPL v2 grants freedoms that have direct consequences: it is why Git can never be taken private, why anyone can audit its security, and why the enormous ecosystem built on top of it — GitHub, GitLab, CI/CD pipelines, and the development workflows of millions of developers — could grow without permission from any single gatekeeper.

The ethical questions open source raises — about profiting from community labor, about who should fund infrastructure software, about where the line falls between using and exploiting — do not have simple answers. But engaging with those questions is part of being a responsible practitioner in this field. Every tool we write in our careers will sit on open-source foundations. Understanding why that happened, and what it means to be part of that tradition, is not optional background knowledge. It is the foundation of professional integrity in software development.

References

- Chacon, S. & Straub, B. Pro Git (2nd ed.). Apress. Available free at: https://git-scm.com/book
- GNU Project. The Free Software Definition. https://www.gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition. https://opensource.org/osd
- Raymond, E.S. The Cathedral and the Bazaar. http://www.catb.org/~esr/writings/cathedral-bazaar
- Torvalds, L. Just for Fun: The Story of an Accidental Revolutionary. HarperBusiness, 2001.
- SPDX License List — GPL-2.0-only. https://spdx.org/licenses/GPL-2.0-only.html
- Git official site. https://git-scm.com